



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



INSTITUTO TECNOLÓGICO
DE CHIHUAHUA

INSTITUTO TECNOLÓGICO DE CHIHUAHUA

Departamento Eléctrica Electrónica

Manual de práctica: Implementación de perceptrón en ESP32 S3

Elaborado por:

Hector Said Herrera Niño	22061074
Abdiel Alejandro Rodríguez Coronado	22061055

Docente: Dr. Juan Alberto Ramírez Quintana

Chihuahua, Chih., lunes 29 de junio de 2026

PRÁCTICA NO. 1

IMPLEMENTACIÓN DE PERCEPTRÓN EN ESP32-S3

OBJETIVO:

Entender el flujo de trabajo típico para implementar un modelo de aprendizaje automático (o inteligencia artificial) en un sistema embebido.

Para ello, se desarrolla una librería escrita en lenguaje C y/o C++ que implemente funciones para generar un dataset de entrenamiento, inicializar un modelo de perceptrón simple, entrenar dicho modelo, probarlo y hacer inferencia en una aplicación. Lo anterior siguiendo la estructura típica de desarrollo de software para sistemas embebidos, es decir, incluyendo capas de abstracción como son HAL y BSP.

MATERIAL Y EQUIPO:

- ESP32-S3
- Cable USB a USB C
- 2 Protoboards
- 2 LEDs
- 1 Pushbutton (Botones pulsadores)
- 5 Potenciómetros o 5 Botones pulsadores
- Cables para conexión

PROCEDIMIENTO:

1. Revisar el programa de ejemplo del directorio "[base_program](#)", el cual incluye un proyecto del IDE Dev-C++ ([link de instalación](#)) que implementa una neurona perceptrón mediante una aplicación de consola simple.

Este código muestra los pasos y algoritmos necesarios para generar un dataset de entrenamiento con compuertas lógicas, crear un modelo perceptrón, entrenarlo mediante el método de "Least Mean Squares" y hacer una prueba con el mismo.

2. Revisar el programa de ejemplo del directorio "[final_program](#)", el cual incluye un proyecto del IDE Arduino ([link de instalación](#)) que implementa el mismo modelo pero en un sistema embebido ([ESP32-S3](#)), con la estructura que se solicita para esta práctica, lo cual se puede ver como un ejemplo de resultado deseado de esta práctica, con ciertas libertades.

3. Desarrollar un programa en Arduino IDE con los siguientes requisitos:

a. Hardware:

- i. Utilizar la tarjeta de desarrollo ESP32-S3.
- ii. Conectar 1 botón pulsador que controle el estado del sistema con dos modos: OFF / RUN.
- iii. Conectar 5 elementos de entrada al sistema, ya sean botones pulsadores o potenciómetros (configurados con un threshold para obtener valores de 0 o 1).
- iv. Conectar 2 elementos de salida al sistema, en este caso 2 LEDs: uno para indicar si el sistema está activo y otro para indicar el estado de la salida del modelo perceptrón.

b. Software de sistema:

- i. El proyecto debe tener de forma abierta el Board Support Package (BSP) y una Hardware Abstraction Layer (HAL) con lo siguiente:

1. HAL:

- HAL.h: Módulo que incluye constantes y variables globales, así como la declaración de tipos de datos y prototipos de funciones para los archivos GPIO.c y ADC.c (este último si se emplean potenciómetros).
- GPIO.c: Componente que contiene la definición de las funciones necesarias para manejar los pines GPIO como entradas y salidas digitales.
- ADC.c: Componente que contiene la inicialización del ADC y la configuración de sus canales (solamente si se termina usando potenciómetros).

2. BSP:

- BSP.h: Módulo que incluye constantes, variables globales, tipos de datos y prototipos de funciones para emplear los recursos de hardware integrados en la tarjeta de desarrollo.
- BSP.c: Componente que contiene la definición de las funciones necesarias para manejar el hardware de la tarjeta de desarrollo.

3. Otros:

- a. Impresión serial (PRINT.h y PRINT.c): Módulo para imprimir mensajes por el puerto serial.
 - ii. El botón para cambiar el estado del sistema debe incluir un algoritmo anti-rebote.
- c. Software de aplicación:
- i. Librería del perceptrón:
 - 1. Se deben crear funciones que hagan lo siguiente:
 - a. Entrenar el modelo.
 - b. Probar el modelo.
 - c. Hacer inferencia con el modelo.
 - 2. El método de implementación de estas funciones y demás recursos (tipos de datos, variables, etc.) queda a consideración del desarrollador.
 - ii. El uso de un RTOS es opcional. Si se emplea, este debe generar los procesos requeridos para el funcionamiento del sistema embebido.
 - iii. Una vez que el sistema se enciende, se deben inicializar los componentes de hardware, se debe entrenar y probar el modelo, y finalmente se debe hacer inferencia con el mismo y las entradas del sistema.
 - iv. El botón para cambiar el estado del sistema debe ser capaz de apagarlo una vez que ya ha sido iniciado.
 - v. El sistema debe ser capaz de apagar o encender el LED de salida dependiendo de la inferencia realizada por el modelo del perceptrón, en base al estado de las entradas.

ENTREGA:

Fecha pendiente.